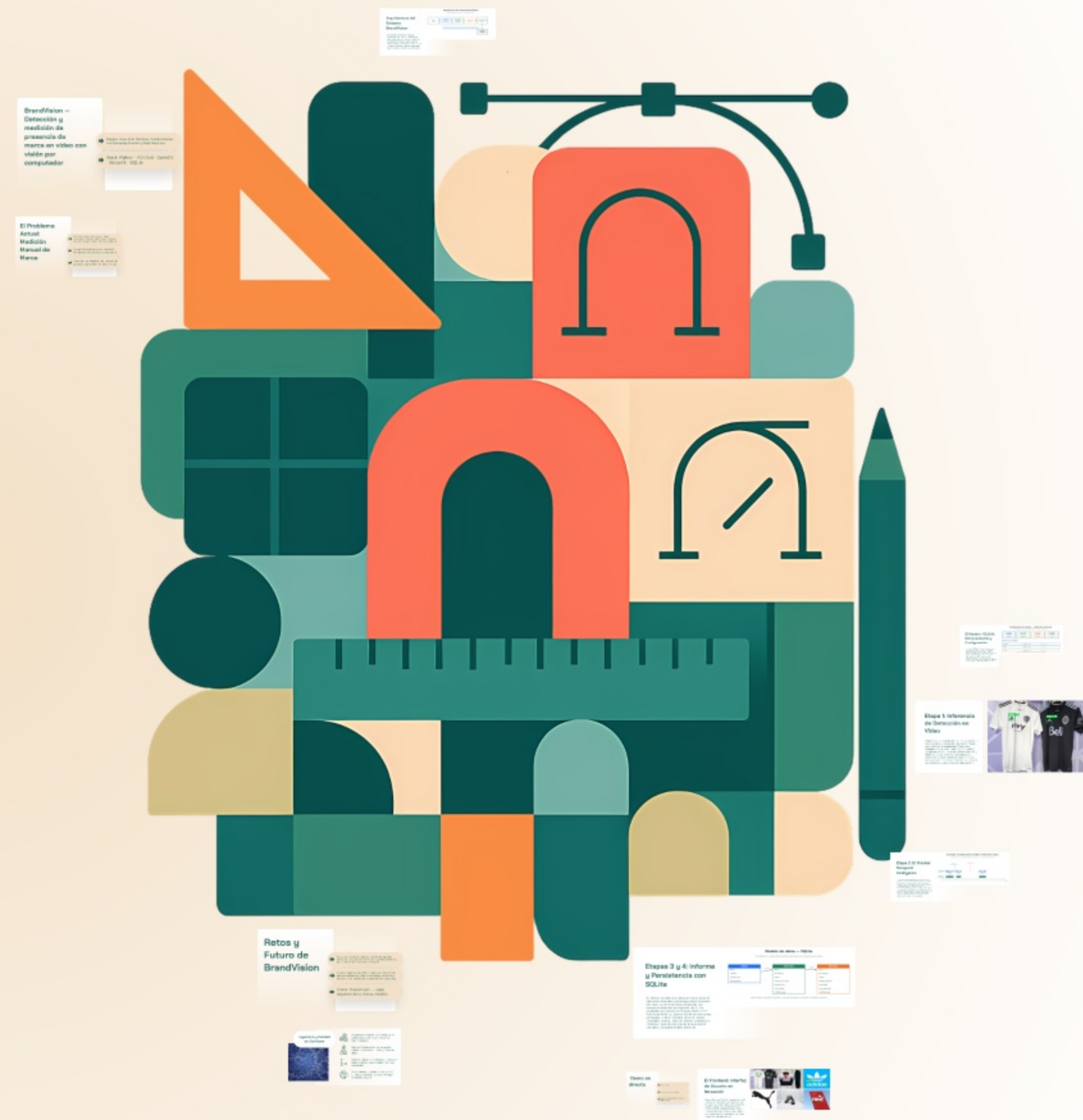


Arquitectura y Funcionamiento Detallado del Sistema



BrandVision – Detección y medición de presencia de marca en vídeo con visión por computador



Equipo: Jose-Julio Ramírez, Camila Arenas,
Iris Fernanda Amorim y Raúl Machaca



Stack: Python · YOLOv8 · OpenCV
· Streamlit · SQLite

El Problema

Actual:

Medición

Manual de

Marca

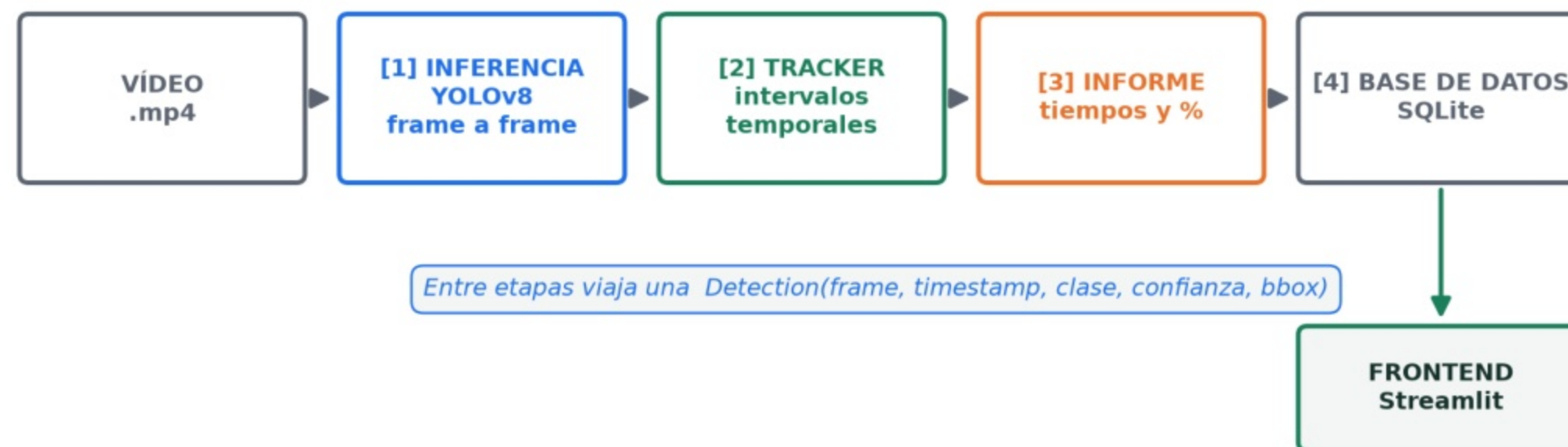
- ➔ Medir la presencia de logos en vídeo (segundos, % de pantalla) se hace de forma manual, resultando lento, costoso y subjetivo.
- ➔ Lo automatizamos con un detector de objetos de visión por computador.
- ➔ Caso de uso: Medición del retorno de patrocinio para Nike, Adidas y Puma.

Arquitectura del Sistema BrandVision

Un pipeline modular de 4 etapas orquestado por `run.py`, procesando vídeo para detectar, rastrear y medir la presencia de marcas. Cada etapa es independiente y testeable, y la estructura de datos `Detection` (frame, timestamp, clase, confianza, bbox) viaja entre ellas.

Arquitectura del sistema BrandVision

Pipeline modular de 4 etapas — orquestado por run.py



El Modelo YOLOv8: Entrenamiento y Configuración

Utilizamos YOLOv8n con transfer learning sobre pesos preentrenados, entrenado para 3 clases (adidas, nike, puma) con un dataset propio etiquetado en Roboflow. Hiperparámetros clave: 50 épocas, imagen de 640px, batch de 16, early stopping (patience 20) y técnicas de data augmentation como volteo horizontal, rotación, variación HSV y mosaic. Las métricas de validación incluyen mAP50, mAP50-95, precisión y recall.

Rendimiento del modelo — validación sobre test

83 imágenes nunca vistas en entrenamiento · 101 logos · umbral 0.40

0.85 mAP@50	0.57 mAP@50-95	0.82 Precisión	0.84 Recall
----------------	-------------------	-------------------	----------------

Detalle por marca (mAP@50)

ADIDAS	mAP@50 = 0.85	P 0.89 · R 0.83
NIKE	mAP@50 = 0.89	P 0.85 · R 0.87
PUMA	mAP@50 = 0.81	P 0.73 · R 0.82

Métricas obtenidas ejecutando model.val() con el best.pt real del proyecto

Etapa 1: Inferencia de Detección en Vídeo

OpenCV se encarga de abrir el vídeo y recorrerlo frame a frame, procesando 1 de cada 3 frames para optimizar el rendimiento (frame_skip configurable). En cada frame, `model.predict()` se ejecuta con un umbral de confianza de 0.40 y NMS con IoU de 0.45. Por cada detección, extraemos la clase, confianza y bounding box (x1,y1,x2,y2), dibujamos la caja con etiqueta y % de confianza, y exportamos un vídeo anotado.

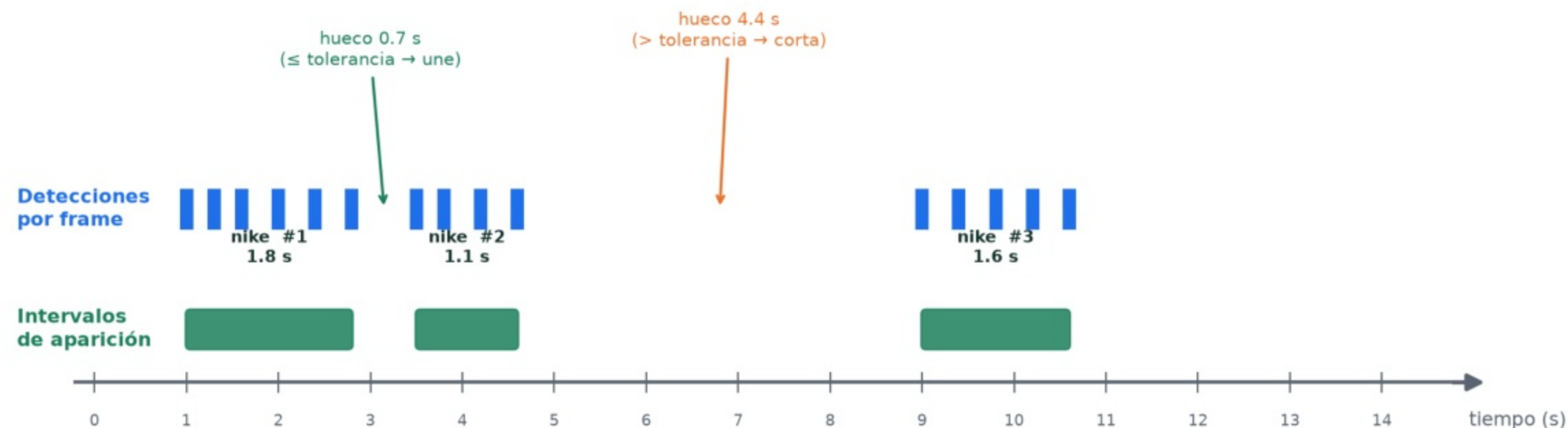


Etapa 2: El Tracker Temporal Inteligente

La inferencia genera detecciones por frame de forma individual; sumarlas directamente sobreestima el tiempo de exposición. Nuestra solución es una máquina de estados por marca que agrupa detecciones consecutivas en `AppearanceInterval` (marca, inicio, fin, nº detecciones, confianza media). Una regla clave es que si el hueco entre dos detecciones de la misma marca es ≤ 1 segundo (`gap\_tolerance`), el intervalo se extiende; si es mayor, el intervalo se cierra y se abre uno nuevo. Al finalizar el vídeo, se cierran los intervalos restantes, midiendo así el tiempo real de exposición.

El tracker: de detecciones sueltas a intervalos reales

Regla: hueco ≤ 1 s $\rightarrow$ extiende el intervalo · hueco > 1 s $\rightarrow$ cierra y abre uno nuevo



Etapas 3 y 4: Informe y Persistencia con SQLite

El informe consolida los datos por marca: duración total de los intervalos, porcentaje sobre la duración del vídeo, y confianza media ponderada. Las marcas sin detección se muestran con 0. Los resultados se exportan en formatos JSON y TXT. Para la persistencia, usamos SQLite con tres tablas principales: `videos` (nombre, duración, fecha), `intervalos` (marca, inicio, fin, frames, confianza) e `informes` (resumen por marca), lo que permite consultar y comparar análisis históricos.

Modelo de datos — SQLite

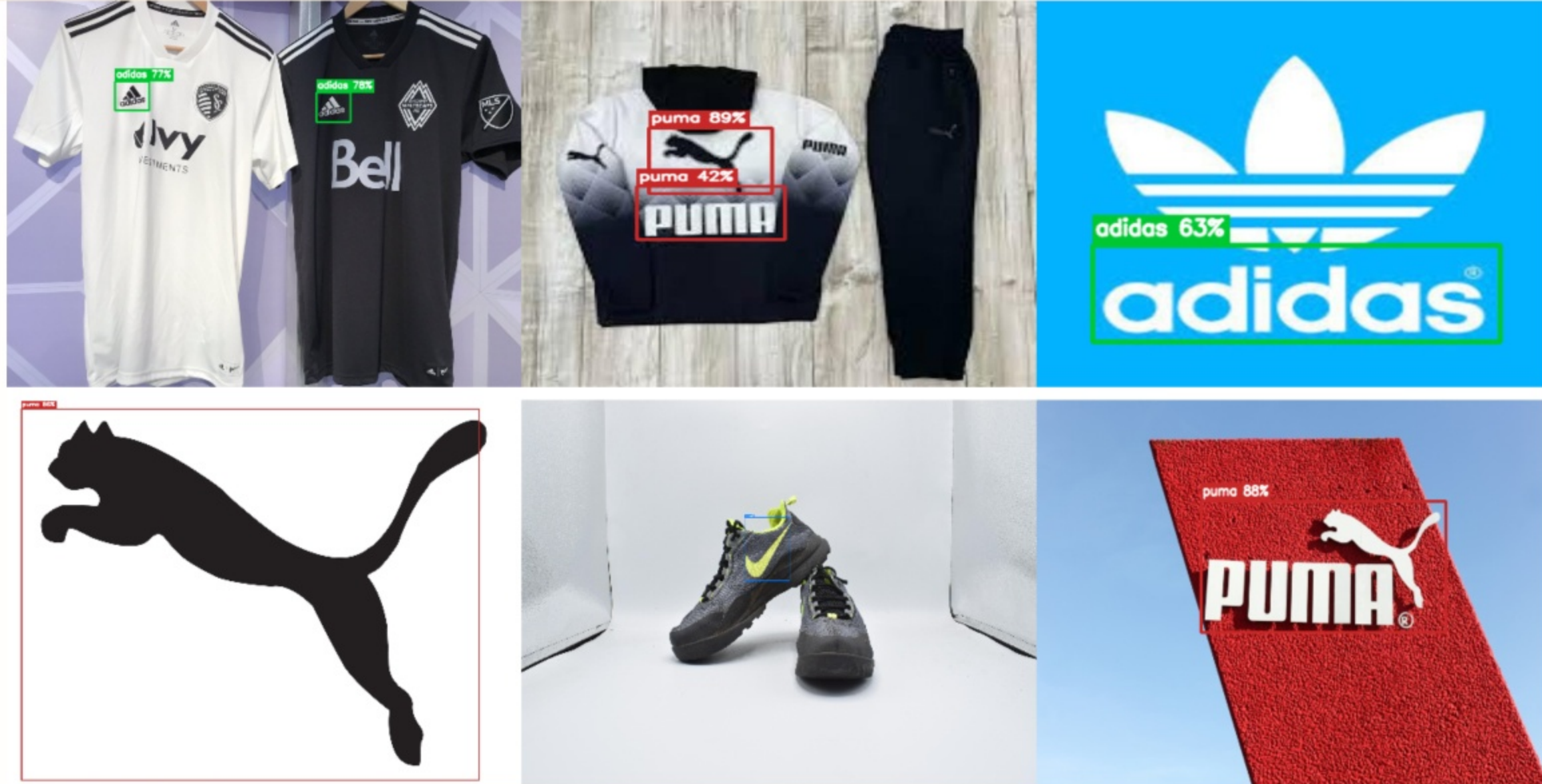
Tres tablas: un vídeo tiene muchos intervalos y un resumen por marca



Cada análisis se persiste completo → permite consultar y comparar campañas históricas

El Frontend: Interfaz de Usuario en Streamlit

Desarrollado en Streamlit, nuestro frontend conecta directamente con el pipeline real. El usuario sube un archivo .mp4, activa el botón 'Analizar', y la aplicación ejecuta todo el proceso. Posteriormente, muestra una tabla resumen por marca (segundos, %), recortes de las detecciones y el vídeo original con las anotaciones.



Demo en directo

➡ Subir vídeo

➡ Ver el análisis en marcha

➡ Explorar resultados (tiempos, %,
detecciones)

Ingeniería y Calidad del Software



Arquitectura modular con configuración centralizada (`config.py` y sin rutas hardcodeadas).



Más de 20 tests unitarios con pytest cubriendo el tracker, informe y base de datos.



Flujo de trabajo Git profesional: ramas por feature, pull requests y tablero Kanban con issues.



Stack robusto y moderno: Python 3.11, Ultralytics YOLOv8, OpenCV, Pandas, Streamlit y SQLite.

Retos y Futuro de BrandVision



Retos: optimización del etiquetado del dataset, equilibrio entre velocidad y precisión (frame_skip), y perfeccionamiento de la medición de tiempo real con el tracker.



Futuras mejoras: expandir el soporte a más marcas, integrar análisis de vídeo en streaming en directo y desarrollar un dashboard comparativo de campañas.



Cierre: BrandVision — cada segundo de tu marca, medido.

BrandVision – Detección y medición de presencia de marca en vídeo con visión por computador

Arquitectura y Funcionamiento Detallado del Sistema



Take this with you. Revisit anytime.

Missed something? Want to explore further?
Scan or click below to open this presentation.
Anytime, anywhere.

[View presentation](#)

